

Data Migration Strategy

Overview

This document outlines the migration strategy for implementing the new data schema in the Chicken Processing System. The migration will be performed in phases to minimize disruption and ensure data integrity.

Current State Analysis

Existing Data Structures

Based on the project files, the following data structures exist:

1. **User models** (`src/models/User.ts`): Basic user interface
2. **Order models** (`src/models/Order.ts`): Basic order interface
3. **Batch models** (`src/models/Batch.ts`): Basic batch interface
4. **Production models** (`src/models/Production.ts`): Production tracking
5. **Compliance models** (`src/models/Compliance.ts`): Compliance checks
6. **Analytics models** (`src/models/Analytics.ts`): Analytics data

New Schema Requirements

The new schema introduces:

1. **Enhanced data structures** with proper relationships
2. **Type-safe interfaces** with validation
3. **Firestore compatibility** with timestamps and references
4. **Business logic enforcement** through schema design

Migration Phases

Phase 1: Schema Preparation (Week 1)

1.1 Database Setup

- Create Firestore database with new collections
- Configure security rules (`firestore.rules`)
- Set up required indexes
- Create backup of existing data

1.2 TypeScript Interfaces

- Implement Firestore schema interfaces (`src/types/firestore-schema.ts`)
- Update existing models to extend new interfaces
- Create data transformation utilities

1.3 Validation Layer

- Implement data validation middleware
- Create schema validation functions
- Set up data integrity checks

Phase 2: Data Migration (Week 2)

2.1 User Data Migration

```
// Migration script for users
async function migrateUsers() {
  const oldUsers = await getOldUsers();
  const newUsers = oldUsers.map(user => ({
    ...transformUser(user),
    createdAt: Timestamp.now(),
    updatedAt: Timestamp.now()
  }));
  await batchWriteToFirestore('users', newUsers);
}
```

Transformation Rules:

- Map old user fields to new schema
- Generate Firebase UID if missing
- Set default account status to 'pending'
- Preserve existing relationships

2.2 Product Data Migration

```
// Migration script for products
async function migrateProducts() {
  const oldProducts = await getOldProducts();
  const newProducts = oldProducts.map(product => ({
    ...transformProduct(product),
    price: convertToMoneyObject(product.price),
    createdAt: Timestamp.now(),
    updatedAt: Timestamp.now()
  }));
  await batchWriteToFirestore('products', newProducts);
}
```

Transformation Rules:

- Convert price strings to Money objects
- Set default stock levels

- Categorize products based on type
- Generate SKU if missing

2.3 Order Data Migration

```
// Migration script for orders
async function migrateOrders() {
  const oldOrders = await getOldOrders();
  const newOrders = oldOrders.map(order => ({
    ...transformOrder(order),
    items: transformOrderItems(order.items),
    createdAt: Timestamp.fromDate(new Date(order.createdAt)),
    updatedAt: Timestamp.now()
  }));
  await batchWriteToFirestore('orders', newOrders);
}
```

Transformation Rules:

- Convert order items to new structure
- Calculate totals using new pricing logic
- Map status values to new enum
- Preserve payment and shipping details

Phase 3: Relationship Migration (Week 3)

3.1 Batch Relationships

- Link products to their production batches
- Update inventory records with batch references
- Set up compliance check relationships

3.2 User Relationships

- Link orders to users with proper references
- Set up user role relationships
- Establish department assignments for staff

3.3 Inventory Relationships

- Connect inventory transactions to products and batches
- Set up expiry tracking
- Establish location relationships

Phase 4: Validation and Testing (Week 4)

4.1 Data Integrity Checks

```
// Validation script
async function validateMigration() {
  const issues = [];

  // Check referential integrity
  issues.push(...await validateUserReferences());
  issues.push(...await validateProductReferences());
  issues.push(...await validateOrderReferences());

  // Check data completeness
  issues.push(...await validateRequiredFields());

  // Check business rules
  issues.push(...await validateBusinessRules());

  return issues;
}
```

4.2 Performance Testing

- Test query performance with new indexes
- Validate security rule enforcement
- Test concurrent access patterns
- Measure read/write latency

4.3 Rollback Preparation

- Create rollback scripts for each collection
- Document rollback procedures
- Test rollback scenarios

Migration Scripts

Main Migration Script

```
// src/scripts/migrate-data.ts
import { migrateUsers } from './migrations/users';
import { migrateProducts } from './migrations/products';
import { migrateOrders } from './migrations/orders';
import { migrateBatches } from './migrations/batches';
import { validateMigration } from './validations';

async function main() {
  console.log('Starting data migration...');

  try {
    // Create backup
    await createBackup();
```

```

// Execute migrations
await migrateUsers();
await migrateProducts();
await migrateOrders();
await migrateBatches();
await migrateInventory();
await migrateCompliance();

// Validate migration
const issues = await validateMigration();

if (issues.length > 0) {
  console.warn(`Found ${issues.length} issues:`);
  issues.forEach(issue => console.warn(`- ${issue}`));

  // Optionally rollback
  if (issues.some(issue => issue.severity === 'critical')) {
    console.error('Critical issues found, initiating rollback...');
    await rollbackMigration();
    process.exit(1);
  }
}

console.log('Migration completed successfully!');

} catch (error) {
  console.error('Migration failed:', error);
  await rollbackMigration();
  process.exit(1);
}
}

```

Data Transformation Utilities

```

// src/utils/data-transform.ts
export function transformUser(oldUser: any): FirestoreUser {
  return {
    id: oldUser.id || generateId(),
    email: oldUser.email,
    firebaseUid: oldUser.firebaseUid || generateFirebaseUid(oldUser.email),
    userType: mapUserType(oldUser.type),
    role: mapUserRole(oldUser.role),
    profile: {
      address: transformAddress(oldUser.address),
      contact: transformContact(oldUser.contact),
      businessName: oldUser.businessName,
      businessRegistrationNumber: oldUser.registrationNumber,
      taxId: oldUser.taxId
    },
  },
}

```

```

    isActive: oldUser.isActive !== false,
    accountStatus: mapAccountStatus(oldUser.status),
    loginAttempts: oldUser.loginAttempts || 0,
    // ... other fields
  };
}

export function transformProduct(oldProduct: any): FirestoreProduct {
  return {
    id: oldProduct.id || generateId(),
    name: oldProduct.name,
    description: oldProduct.description || '',
    type: mapProductType(oldProduct.type),
    price: {
      amount: parseFloat(oldProduct.price) || 0,
      currency: 'ZAR'
    },
    stockQuantity: oldProduct.stock || 0,
    minStockLevel: oldProduct.minStock || 10,
    maxStockLevel: oldProduct.maxStock || 100,
    // ... other fields
  };
}

```

Rollback Strategy

Rollback Triggers

1. **Data corruption** detected during validation
2. **Performance degradation** beyond acceptable limits
3. **Security issues** identified post-migration
4. **Business logic failures** in critical operations

Rollback Procedures

```

// src/scripts/rollback-migration.ts
async function rollbackMigration() {
  console.log('Initiating rollback...');

  // Restore from backup
  await restoreBackup();

  // Revert schema changes
  await revertSchemaChanges();

  // Update application configuration
  await updateAppConfig('old');

  console.log('Rollback completed successfully');
}

```

Backup Strategy

- **Full backup** before migration starts
- **Incremental backups** during migration
- **Point-in-time recovery** capability
- **Off-site storage** for disaster recovery

Testing Strategy

Unit Tests

- Test data transformation functions
- Test validation logic
- Test migration scripts

Integration Tests

- Test Firestore operations
- Test security rules
- Test data relationships

Performance Tests

- Load testing with migrated data
- Query performance testing
- Concurrent access testing

User Acceptance Testing

- Business user verification of data
- Application functionality testing
- Report generation validation

Timeline

Week 1-2: Preparation

- Complete schema design
- Develop migration scripts
- Set up test environment
- Train team members

Week 3: Dry Run

- Execute migration in test environment
- Validate results
- Optimize performance

- Fix identified issues

Week 4: Production Migration

- Schedule maintenance window
- Execute production migration
- Validate production data
- Monitor system performance

Week 5: Post-Migration

- Performance optimization
- User training
- Documentation updates
- Lessons learned review

Risk Mitigation

Technical Risks

1. **Data loss:** Mitigated by comprehensive backups
2. **Performance issues:** Mitigated by performance testing
3. **Integration failures:** Mitigated by thorough testing

Business Risks

1. **Downtime:** Scheduled during low-traffic periods
2. **User disruption:** Communicated in advance
3. **Data inaccuracies:** Validated through multiple checks

Operational Risks

1. **Team availability:** Cross-trained team members
2. **Vendor dependencies:** Coordinated with service providers
3. **Regulatory compliance:** Verified with legal team

Success Criteria

Technical Success

- All data migrated without loss
- Performance meets or exceeds requirements
- Security rules properly enforced
- System stability maintained

Business Success

- Zero downtime for critical operations
- Users can access all required functionality

- Reports generate accurately
- Business processes continue uninterrupted

Operational Success

- Team members trained on new system
- Documentation updated and accurate
- Support processes established
- Monitoring and alerting configured

Conclusion

This migration strategy provides a comprehensive plan for transitioning to the new data schema while minimizing risk and ensuring data integrity. The phased approach allows for thorough testing and validation at each step, with clear rollback procedures in case of issues.